

NF1: Almacenamiento y Gestión de Datos

2.1 ¿Qué es Big Data? Las 4 Vs

El Big Data no es solo un gran volumen de información; es aquel conjunto de datos que supera la capacidad de gestión de las herramientas tradicionales. Se define por 4 dimensiones críticas:

1. **Volumen:** La cantidad masiva de datos generados cada segundo (ej. todos los sensores de una *Smart City*). 
2. **Velocidad:** El flujo de datos es un "vídeo en directo", no una foto fija. Llegan sin interrupción 24/7 y requieren procesamiento inmediato. ⚡
3. **Variedad:** Los datos no son homogéneos. Mezclamos tablas (estructurados), texto/fotos de redes sociales (no estructurados) y lecturas de sensores. 🌈
4. **Veracidad:** La lucha por la calidad y fiabilidad. Implica gestionar datos erróneos o ruidosos (ej. un sensor de temperatura estropeado). Es el gran reto técnico. 🔍

2.2 Data Lake vs. Data Warehouse

Para almacenar la información de una arquitectura de datos, existen dos estrategias principales:

- **Data Warehouse (La Biblioteca)** 
 - **Concepto:** Almacenamiento muy ordenado y etiquetado.
 - **Esquema: Schema-on-Write.** El dato debe ser catalogado y transformado *antes* de entrar.
 - **Característica:** Es rígido. Si el dato no encaja en la estructura previa, no se puede guardar fácilmente.
- **Data Lake (El Almacén de LEGOs)** 
 - **Concepto:** Repositorio gigante donde se guardan los datos en su **formato original** (CSV, JSON, imágenes, etc.).
 - **Esquema: Schema-on-Read.** El orden y la estructura se aplican solo cuando vas a usar el dato ("aplicas el plano al construir").
 - **Ventaja:** Ideal para IA y Smart Cities por su flexibilidad ante necesidades de análisis futuras desconocidas.

2.3 Formatos de Almacenamiento: CSV, JSON y Parquet

La elección del formato determina el **coste** (almacenamiento) y la **velocidad** (procesamiento).

- **CSV:** Simple y universal, pero **pesado y frágil**. No tiene metadatos claros y no comprime bien. 📁

- **JSON:** Estructura jerárquica, ideal para APIs. Sigue siendo formato texto, por lo que ocupa mucho espacio (voluminoso). 🌐
- **Parquet (El Estándar de la Industria) 🏆 :**
 - **Formato Binario:** Mucho más ligero y comprimido.
 - **Almacenamiento Columnar:** A diferencia del CSV (que lee por filas), Parquet organiza los datos por columnas.
 - **Ventaja Clave:** Si solo necesitas una columna específica (ej. "velocidad media"), el sistema lee solo esa parte del archivo, ignorando el resto. Esto lo hace extremadamente **rápido y barato** para analítica avanzada.

2.4 Comparativa de Soluciones de Almacenamiento 📁

En Big Data, no hay una herramienta "mejor", sino una **adecuada para cada problema**. Aquí tienes la comparativa de las tres grandes familias:

Criterio	SQL (Relacional) 🏛️	Data Lake (Distribuido) 🌊	NoSQL (Documental) 📄
Analogía	Biblioteca con fichas exactas.	Almacén de materias primas.	Archivador de carpetas por proyecto.
Prioridad	Consistencia e integridad.	Flexibilidad a gran escala.	Velocidad y evolución rápida.
Esquema	Schema-on-Write (Rígido).	Schema-on-Read (Flexible).	Dinámico (Cambia fácilmente).
Escalado	Vertical (Máquina más potente).	Horizontal (Más máquinas/nodos).	Horizontal (Reparte la carga).
Uso Ideal	OLTP: Transacciones (Bancos).	OLAP: Análisis masivo e IA.	High-Throughput: Apps móviles.

La Caja de Herramientas del Ingeniero de Datos

Para tu examen o práctica, quédate con estas conclusiones de uso según el escenario:

1. **¿Necesitas integridad y transacciones seguras?** (Ej: Facturación, banca, CRMs)
 -  **Usa SQL.** Su fuerte son las relaciones complejas entre tablas.
2. **¿Necesitas guardar de todo para entrenar modelos de IA o hacer BI histórico?** (Ej: Logs, sensores, imágenes)
 -  **Usa Data Lake.** Es el corazón del Big Data (usa Spark y Parquet). *Ojo con el "Data Swamp" (pantano de datos) si no hay orden.*
3. **¿Necesitas que tu app web/móvil vuele y los datos cambian a menudo?** (Ej: Perfiles de usuario, catálogos e-commerce)
 -  **Usa NoSQL (MongoDB).** Ideal para leer documentos completos muy rápido sin complicarse con JOINS.

Limitaciones que debes recordar

- **SQL:** Sufre con datos no estructurados y es caro de escalar.
- **Data Lake:** No sirve para actualizar un solo registro rápido o consultas de baja latencia.
- **NoSQL:** No es bueno para realizar consultas que crucen mucha información distinta (JOINS complejos).